

Learn2Slither

Plan de developpement - Q-learning sur un Snake 10x10

Projet 42 de reinforcement learning. Un agent apprend a jouer au Snake par Q-learning, en ne percevant que ce que le serpent voit dans les quatre directions depuis sa tete (vision en rayons). Ce document decoupe le travail en phases livrables, avec les points de vigilance qui concentrent les penalites a l'evaluation.

En bref

Modele autorise	Q-fonction uniquement (Q-table ou reseau de neurones)
Contrainte cle	vision limitee aux 4 rayons depuis la tete
Livrables modeles	1, 10 et 100 sessions minimum
Objectif	longueur ≥ 10 et duree de vie maximale
Normes	flake8 propre, jamais de crash

1. Architecture cible

Structure modulaire imposee par le sujet. Flux de donnees a sens unique : **Environment -> Interpreter (state + reward) -> Agent (action) -> Environment**. Aucune logique d'apprentissage hors de l'agent, aucune information de board hors de l'interpreter.

```
learn2slither/  
snake          # point d'entree, parsing CLI, boucle d'entrainement  
environment.py  # board 10x10, regles du jeu (aucun apprentissage)  
interpreter.py  # calcule l'etat (vision) et les rewards  
agent.py       # Q-learning : Q-table, epsilon-greedy, update  
display.py     # interface pygame : vitesse + step-by-step  
models/        # modeles sauvegardes (1sess, 10sess, 100sess...)  
requirements.txt # pygame, numpy, flake8
```

Responsabilites par module

- **environment.py** : etat du board, deplacement du serpent, apparition des pommes, detection de game over. Ne connait ni state ni reward.
- **interpreter.py** : traduit le board en vision (4 rayons) puis en features compactes, et calcule la recompense de la transition.
- **agent.py** : Q-table `{state: [q\_up, q\_left, q\_down, q\_right]}`, politique epsilon-greedy, regle de mise a jour, save/load JSON.
- **display.py** : rendu pygame, vitesse configurable, mode pas a pas ; totalement optionnel via `-visual off`.

2. Points de vigilance (penalites)

Vision limitee - penalite -42

L'agent ne recoit QUE ce que le serpent voit dans les 4 directions depuis sa tete. Ne jamais lui passer les coordonnees absolues, la grille complete, ou la position des pommes hors des rayons.

Modele impose - note 0 sinon

Uniquement une Q-fonction (Q-table ou NN). Aucun autre modele d'apprentissage n'est autorise.

Robustesse - crash = 0

Le programme ne doit jamais crasher, quelle que soit l'entree CLI ou le fichier modele charge. Prevoir la gestion d'erreurs des le depart.

Autres exigences bloquantes : affichage terminal de la vision avant chaque action (caracteres W mur, H tete, S corps, G pomme verte, R pomme rouge, O vide) puis l'action choisie ; et code passant **flake8** sans aucune erreur.

3. Plan par etapes

Etape 0 - Mise en place

Squelette du projet, venv, requirements, entree `snake` executable. Definir les constantes (taille board, actions, caracteres d'affichage).

- Creer les 5 modules vides avec leurs signatures.
- requirements.txt : pygame, numpy, flake8.
- Verifier `flake8` passe sur le squelette.

Etape 1 - Environment

Board 10x10, serpent initial de 3 cellules contigues place aleatoirement, 2 pommes vertes + 1 rouge. Deplacement, consommation, conditions de game over.

- Placement aleatoire valide du serpent et des pommes.
- Pomme verte : +1 longueur + respawn ; rouge : -1 + respawn.
- Game over : mur, collision queue, ou longueur 0.
- Affichage terminal du board pour debug.

Etape 2 - Interpreter (vision + reward)

Extraire les 4 rayons depuis la tete, les reduire en features compactes et independantes de la taille du board. Calculer la recompense.

- Par direction : danger adjacent, pomme verte visible, pomme rouge visible.
- Encodage de l'etat en cle hashable pour la Q-table.
- Rewards ajustables : vert +20, rouge -20, rien -1, game over -100.
- Affichage terminal de la vision (W/H/S/G/R/O).

Etape 3 - Agent Q-learning

Q-table en dictionnaire, politique epsilon-greedy avec decay, regle de mise a jour, serialisation JSON.

- $Q(s,a) += \alpha * (r + \gamma * \max_{a'} Q(s')) - Q(s,a)$.
- $\alpha \approx 0.1$, $\gamma \approx 0.9$, epsilon 1.0 -> 0.01 avec decay.
- `-dontlearn` : epsilon = 0 et aucune mise a jour.
- save/load : un fichier unique contenant tout l'etat d'apprentissage.

Etape 4 - Boucle & CLI

Assembler la boucle d'entrainement et le parsing des flags. Afficher le bilan de fin de session.

- Flags : -sessions, -save, -load, -visual on|off, -dontlearn, -step-by-step.
- Boucle : Environment -> Interpreter -> Agent -> Environment.
- Fin de session : `Game over, max length = X, max duration = Y`.

Etape 5 - Display pygame

Rendu graphique optionnel, vitesse réglable, mode pas a pas. Aucune dependance de la logique de jeu vis-a-vis de l'affichage.

- Rendu board + serpent + pommes.
- Vitesse configurable ; `step-by-step` avance action par action.
- `visual off` pour accelerer l'entrainement.

Etape 6 - Entrainement & livraison

Entraîner et sauvegarder les modeles demandes, valider les criteres.

- Generer models/1sess, 10sess, 100sess (et 1000sess).
- Verifier longueur ≥ 10 en exploitation (`-dontlearn`).

Passez à la fin de l'ère 8 tests anti-crash sur entrees invalides.

4. Reperes techniques

Q-learning

```
Q(s,a) += alpha * (r + gamma * max(Q(s')) - Q(s,a))

alpha  = 0.1      # taux d'apprentissage
gamma  = 0.9      # facteur d'actualisation
eps    = 1.0 -> 0.01 (epsilon-greedy avec decay)
```

Encodage de l'etat

Reduire chaque rayon en features compactes (danger, pomme verte, pomme rouge par direction). Un etat independant de la taille du board ouvre le bonus *board size variable*. La Q-table est un dict serialise en JSON.

Interface CLI attendue

```
./snake -sessions 10 -save models/10sess.txt -visual off
./snake -visual on -load models/100sess.txt -sessions 10 \
    -dontlearn -step-by-step
```

5. Bonus (si le mandatory est parfait)

- Longueur atteinte 15 / 20 / 25 / 30 / 35.
- Affichage avance : lobby, panneau de configuration, statistiques.
- Taille de board paramettable, meme modele fonctionnel sur toute taille.

Ordre de developpement conseille : Environment -> Interpreter -> Agent -> boucle CLI, en validant flake8 et l'absence de crash a chaque etape avant de passer a la suivante.